CS 362 / 562 - Assignment 4 Spring, 2026

Assignment 4 - Data Challenge

Hosts and guests on Airbnb face a common problem: how should a listing be priced? This data challenge asks you to answer that question using real listing data from one or more Airbnb markets. You are given a training dataset with known prices and a test dataset without prices. Your goal is to build a model that predicts the nightly price of a listing as accurately as possible and submit those predictions as a CSV file.

This data challenge requires you to:

- Complete all parts of the implementation
- Answer the reflection questions (graduate students)

This challenge is designed to take approximately two hours, but it is not timed. Accept this assignment at: https://classroom.github.com/a/YW3LeEsA

Implementation

There are four parts to this assignment:

- EDA / Data Exploration
- Model construction — with a performance requirement
- Predictions

Each of these parts is explained in this section. You will need the two CSV files linked from the GitHub repository:

File	Description
listings_train.csv	Training data. Contains all features plus the target column price. Use this file to build and evaluate your model.
listings_test_features.csv	Test data. Contains all features but no price column. Generate one prediction for each row and submit as predicted_price.csv.

Table 1: Files available for this Data Challenge

Your task in this data challenge is to predict one price for each listing in the `listing_test_features.csv` file.

Implementation: EDA / Data Exploration

Answer each question below. Support your answer with a chart or printed output.

#	Questions	Hint (if any)
0: Framing	What type of problem is this — supervised / unsupervised? Classification, clustering, regression)?	This is a regression problem
1:	How many columns are there in the training set? How many in the test set? Where do they match / mismatch? Which contain missing values? How many rows are there in the training set? How many in the test set? How many are usable as is?	There are 36 columns and 42234 rows in the training set and 35 columns with 8199 rows in the test set; the test set has no price column and Training set - Missing values (Notna count): Bathrooms (42192), Bedrooms (42187), Beds (42154). Last_review (33099), reviews_per_month (33099), review_scores_rating (33099), review_scores_accuracy (33098), review_scores_cleanliness (33098), review_scores_checkin (33097), review_scores_communication (33098), review_scores_location (33097), review_scores_value (33097), license (0) Test set - Missing values (Notna count): Neighbourhood_group (0), license (0)
2:	Distributions Is price normally distributed? What is the median price? How are features distributed — normally / not normally? How many unique values does each non-numeric column (eg. neighbourhood) have? How will you handle these columns?	Train_df median price is 141.0 Price is not normally distributed - it's right skewed. Name: 41118 (unique listing titles) Host_name: 9092 (first) Host_has_profile_pic: 3 (y, n, 16 Nan) Host_is_verified: 3 (y, n, Nan) Neighbourhood: 33 (uk) Room_type: 4 (entire home/apt, private room, shared room, hotel room) Amenities: 38642 (unique list of amenities) Last_review: 1661 (dates, 9135 Nan) Of these columns I'll drop the Name, host_has_profile_pic. And convert the last_review date to a numeric format (probably by year but converting year-month to Year.month numerically shouldn't be too hard. (ie: if the x-axis was incrementally by year starting from the oldest airbnb to the newest, there'd be 12 hatch-marks between each year to indicate month) I don't think it'll happen but with transforming last-review date we could track a small portion of inflation.

3:	CS 562 Is there a relationship between the price and minimum_nights columns? Consider a histplot for this. Consider a pairplot for this.	Visually, it doesn't seem that way, although there's a noticeable density of listings where minimum nights are fewer and the listing is cheaper. Although that doesn't entirely indicate correlation or causation. Most if not all of the listings have a stay of around 1 night minimum give or take, but vary in price range.

Graph Created for Q3 ^

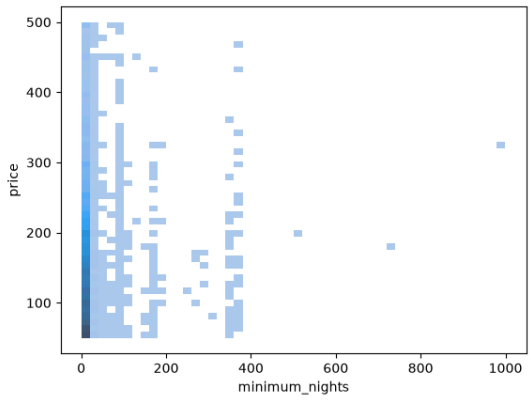


Table 2: Questions for EDA

Implementation: Model Construction

Using the training file, build a model to predict the nightly price of an Airbnb listing. You may use any or all of the following algorithms covered in class: Linear Regression, CART (Decision Tree), K-Means, and PCA. You may use other algorithms such as Random Forests, AdaBoost / XGBoost, or neural networks but you must be able to explain how any other algorithm works to the satisfaction of the grader.

This dataset is imperfect, as are many datasets in the real world. You may find some of the following useful:

- Drop low-correlation columns
- One-hot encoding using sklearn (eg. here) — because it can subsequently be applied to the test set)
- Handling missing values

Build the model exclusively in the Jupyter notebook provided in the repository. The model construction has the following requirements:

- Your notebook must run end-to-end without errors with fully-executed cells.
- Structure your notebook into cells of reasonable scope. Each major step (loading data, cleaning, feature engineering, model training, evaluation) should be clearly separated.

- Use **train_test_split** on the training data to develop and compare models internally. Do not use the test features file for model selection.
- Report RMSE for each model you try, evaluated on your internal validation split.

Implementation: Predictions

Be careful to apply any transformations on the training set, such as one-hot encoding, to the test set as well. Once your final model is trained, generate predictions for every row in **listings_test_features.csv** using something similar to:

```
predictions = trained_model.precict(test_df)
```

and save them to a file named **predicted_price.csv** using something similar to:

```
pd.DataFrame(predictions).to_csv('A4_predictions_MYNAME.csv',
index=False)
```

… replacing MYNAME with your name. The file must contain a single column with one prediction per row in the test file, for example:

price
120.00
85.50
210.00
…

Your model will be evaluated against the actual test prices (which you do not have). The RMSE (root mean squared error) thresholds below define your performance grade for Implementation.

Grade	CS 362 (Undergrad)	CS 562 (Grad)
Exceptional	RMSE < 65	RMSE < 63
Good	RMSE between 65 and 85	RMSE between 63 and 80
Partial	RMSE ≥ 85	RMSE ≥ 80

Table 3: Performance thresholds

These thresholds are achievable using only the algorithms covered in class

Reflection

Graduate students are required to perform this reflection upon completion of the Implementation above. Thoughtful, specific answers that reference your actual results are valued over generic ones.

1. Which features were most predictive of nightly price? Were the results what you expected, or were you surprised by anything?
2. What does your model reveal about these Airbnb markets? For example: does neighbourhood matter more than property size?
3. What additional data would have improved your model? Name at least one variable not present in the dataset that you believe would be a strong predictor of price, and explain why.

Submission

A starting repository (containing a skeleton notebook and links to the data files) is provided for this assignment. Complete all work in the notebook provided. Including answers to EDA / Data Exploration questions as well as to Reflection questions (for graduate students).

Grading

Once you submit your implementation, an evaluator will determine whether your implementation passes correctness and other quality standards, as detailed in the bullets below, along with the RMSE of your model. If your implementation meets or exceeds these standards, the evaluator may invite you to schedule a code review.

Assignment grades are based on elements of quality standards are as follows:

- Implementation (3 sections)
    - EDA / Data Exploration — your work must show an understanding of the data and data quality
    - Model construction — the model must be built with care and attention to detail along standards in class
    - Predictions — your predictions must meet the expectations for output format ● Model performance — your model's performance as indicated by Table 3.
- Reflection: your answers to the reflection questions must indicate that you have engaged with the questions in a sophisticated manner.
- Style: in the eyes of the evaluator, your implementation should be well-commented and must consistently use intelligently-named variables and functions.
- Documentation: in the eyes of the evaluator, the github repository must be well-documented and must not contain extraneous data, files or information.
- Code review: graduate students with passing implementation and efficiency may be invited to a code review session to review the submission. Some undergraduate students may also be required to perform a code review. In some cases, your evaluator will be unable to provide a grade until after the code review has been reviewed by the instructor or a second evaluator.

Your grade for this project will be determined as follows:

	Exceptional	**Good**	**Partial**
Reflection	*Required*	*Required*	*Submission*

EDA	*Required*	*Required*	*contains one or more of these elements, but does not contain all of them*
Model construction	*Required*	*Required*	
Predictions	*Required*	*Required*	
Model performance	*Good or higher*	*Partial or higher*	
Style	*Required*		

Documentation	*Required*		
Code Review	*Required (CS 562)* *Possible (CS 362)*	*Required (CS 562)* *Possible (CS 362)*	*Not offered*

Missing submissions receive an Unassessable grade.

Late submissions and resubmissions

Submissions are allowed past the due date until the solution has been posted. Any late submission costs one (1) token.

Once submissions are graded, resubmissions are permitted until the solution has been posted. Each resubmission costs one (1) token.

Use of LLMs and the code review

For this assignment, you are permitted the use code assist tools or large language models (LLMs) for the implementation portion. This may be especially useful for making your code more “pythonic” or more efficient. If you use an LLM, you must answer the following questions:

- Which LLM(s) did you use?
- What was your series of prompts?
- Did you find this use helpful?
- What did you learn from its use?

If you use a code assist tool or LLM:

1. You must clearly indicate what items are not exclusively your own work. In other words, you must cite any external sources of work (eg. StackOverflow posts by URL) and you must indicate code you use which was generated by an LLM or other tool. This indication may be in the form of a comment.
2. You are responsible for everything you submit. In other words, you must clearly understand the design, implementation and style choices of all the project artifacts (code, comments, etc.) you submit.

Practically, this means that the instructor or TA may require that you thoroughly explain the design, implementation or style choices during a code review. These code reviews are required for graduate students enrolled in CS 562 but may also be required for some undergraduate students.